

Advanced Mockito Hands-On Exercises

MockitoExercises

```
|
|— src/main/java
|   |— Repository.java
|   |— Service.java
|   |— RestClient.java
|   |— ApiService.java
|   |— MyFileReader.java
|   |— MyFileWriter.java
|   |— FileService.java
|   |— NetworkClient.java
|   └─ NetworkService.java
|
└─ src/test/java
    |— ServiceTest.java
    |— ApiServiceTest.java
    |— FileServiceTest.java
    |— NetworkServiceTest.java
    └─ MultiReturnServiceTest.java\
```

Repository.java

```
public interface Repository {
    String getData();
}
```

Service.java

```
public class Service {

    private Repository repository;

    public Service(Repository repository) {
        this.repository = repository;
    }

    public String processData() {
        return "Processed " + repository.getData();
    }
}
```

RestClient.java

```
public interface RestClient {  
    String getResponse();  
}
```

ApiService.java

```
public class ApiService {  
  
    private RestClient restClient;  
  
    public ApiService(RestClient restClient) {  
        this.restClient = restClient;  
    }  
  
    public String fetchData() {  
        return "Fetched " + restClient.getResponse();  
    }  
}
```

MyFileReader.java

Don't use Java's built-in FileReader because Mockito cannot mock it easily.

```
public interface MyFileReader {  
    String read();  
}
```

MyFileWriter.java

```
public interface MyFileWriter {  
    void write(String content);  
}
```

FileService.java

```
public class FileService {  
  
    private MyFileReader fileReader;  
    private MyFileWriter fileWriter;  
  
    public FileService(MyFileReader fileReader,  
        MyFileWriter fileWriter) {  
        this.fileReader = fileReader;  
        this.fileWriter = fileWriter;  
    }  
}
```

```

    }

    public String processFile() {
        String content = fileReader.read();
        fileWriter.write(content);
        return "Processed " + content;
    }
}

```

NetworkClient.java

```

public interface NetworkClient {
    String connect();
}

```

NetworkService.java

```

public class NetworkService {

    private NetworkClient networkClient;

    public NetworkService(NetworkClient networkClient) {
        this.networkClient = networkClient;
    }

    public String connectToServer() {
        return "Connected to " + networkClient.connect();
    }
}

```

Test Classes

ServiceTest.java

```

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.*;

import org.junit.jupiter.api.Test;

public class ServiceTest {

    @Test
    public void testServiceWithMockRepository() {

```

```
Repository mockRepository = mock(Repository.class);

when(mockRepository.getData())
    .thenReturn("Mock Data");

Service service = new Service(mockRepository);

String result = service.processData();

assertEquals("Processed Mock Data", result);
}
}
```

ApiServiceTest.java

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.*;

import org.junit.jupiter.api.Test;

public class ApiServiceTest {

    @Test
    public void testServiceWithMockRestClient() {

        RestClient mockRestClient = mock(RestClient.class);

        when(mockRestClient.getResponse())
            .thenReturn("Mock Response");

        ApiService apiService =
            new ApiService(mockRestClient);

        String result = apiService.fetchData();

        assertEquals("Fetched Mock Response", result);
    }
}
```

FileServiceTest.java

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.*;
```

```
import org.junit.jupiter.api.Test;

public class FileServiceTest {

    @Test
    public void testServiceWithMockFileIO() {

        MyFileReader mockFileReader =
            mock(MyFileReader.class);

        MyFileWriter mockFileWriter =
            mock(MyFileWriter.class);

        when(mockFileReader.read())
            .thenReturn("Mock File Content");

        FileService fileService =
            new FileService(
                mockFileReader,
                mockFileWriter);

        String result =
            fileService.processFile();

        assertEquals(
            "Processed Mock File Content",
            result);

        verify(mockFileWriter)
            .write("Mock File Content");
    }
}
```

NetworkServiceTest.java

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.*;

import org.junit.jupiter.api.Test;

public class NetworkServiceTest {
```

```

@Test
public void testServiceWithMockNetworkClient() {

    NetworkClient mockNetworkClient =
        mock(NetworkClient.class);

    when(mockNetworkClient.connect())
        .thenReturn("Mock Connection");

    NetworkService networkService =
        new NetworkService(
            mockNetworkClient);

    String result =
        networkService.connectToServer();

    assertEquals(
        "Connected to Mock Connection",
        result);
}
}

```

MultiReturnServiceTest.java

```

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.*;

import org.junit.jupiter.api.Test;

public class MultiReturnServiceTest {

    @Test
    public void testServiceWithMultipleReturnValues() {

        Repository mockRepository =
            mock(Repository.class);

        when(mockRepository.getData())
            .thenReturn("First Mock Data")
            .thenReturn("Second Mock Data");
    }
}

```

```
Service service =  
    new Service(mockRepository);  
  
String firstResult =  
    service.processData();  
  
String secondResult =  
    service.processData();  
  
assertEquals(  
    "Processed First Mock Data",  
    firstResult);  
  
assertEquals(  
    "Processed Second Mock Data",  
    secondResult);  
}  
}
```

Maven Dependencies (pom.xml)

If you're using a Maven project in Eclipse, add:

```
<dependencies>
```

```
    <dependency>
```

```
        <groupId>org.junit.jupiter</groupId>
```

```
        <artifactId>junit-jupiter</artifactId>
```

```
        <version>5.12.2</version>
```

```
        <scope>test</scope>
```

```
    </dependency>
```

```
    <dependency>
```

```
        <groupId>org.mockito</groupId>
```

```
        <artifactId>mockito-core</artifactId>
```

```
        <version>5.18.0</version>
```

```
        <scope>test</scope>
```

```
    </dependency>
```

```
</dependencies>
```

- MockitoExercises [boot]
- src/main/java
 - com.example.mockito
 - ApiService.java
 - FileService.java
 - MockitoExercisesApplication.java
 - MyFileReader.java
 - MyFileWriter.java
 - NetworkClient.java
 - NetworkService.java
 - Repository.java
 - RestClient.java
 - Service.java
- src/test/java
 - com.example.mockito
 - ApiServiceTest.java
 - FileServiceTest.java
 - MultiReturnServiceTest.java
 - NetworkServiceTest.java
 - ServiceTest.java















